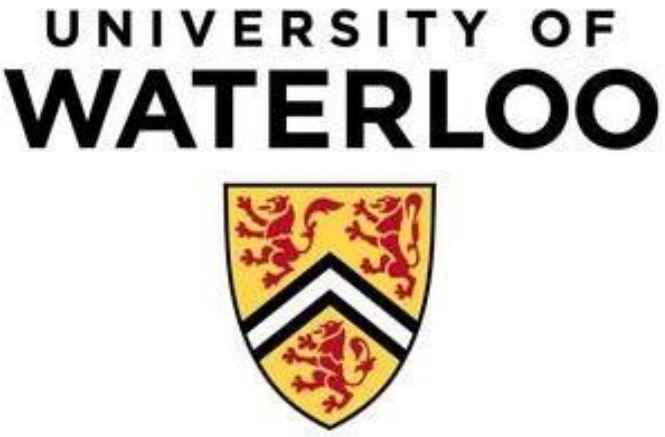
MSCI 623 PROJECT

Air Quality Analysis and Prediction using Machine Learning



Submitted by:

Anjali Anadkat (20861897)
Yogda Gharat (20848008)

Guided by:

Professor Lukasz Golab

ABSTRACT

Owing to the global increase in industrialisation and urbanisation, Air Pollution has become a major cause of concern today. The detrimental effects of outdoor and indoor air pollution have not only resulted in chronic diseases but it is also the reason for millions of pre-mature deaths around the world. It has increasingly become crucial to monitor and control the air quality so as to reduce its harmful effects to human health and the environment. In order to enable individuals to make informed decisions based on the air quality data, it is important to predict the Air Quality Index (AQI) and the factors affecting it. This study focuses on using Supervised Learning techniques to predict the AQI_bucket, a categorical variable which is easy to understand and comprehend as compared to the numeric AQI values. The classifiers highlight the air pollutants that are most likely to influence the AQI_bucket. The techniques are compared based on their prediction accuracy. Moreover, K-means clustering – an unsupervised learning technique has been implemented in order to show clusters based on concentrations of air pollutants. This study also discusses the impact of Covid19 lockdown on the level of air pollution and provides future recommendations which can be useful to environment regulators and policy makers.

1. INTRODUCTION

Air pollution (both indoor and outdoor pollution) constitutes the most persistent environmental health risk facing the global population in the history of time. According to the World Health Organization, 7 million people are at health risk due to air pollution [1].

There are two main types of air pollution – ambient air pollution (outdoor pollution) which arises from industrial and vehicular emissions; and household (or indoor) air pollution which refers to pollution generated by household combustion of fuels such as coal, wood or kerosene while using open fires or basic stoves in poorly ventilated spaces. Women and children, who tend to spend more time indoors, are affected the most.

The main air pollutants which are detrimental to health include (i) particulate matter, a mix of solid and liquid droplets arising mainly from fuel combustion and road traffic; (ii) nitrogen dioxide from road traffic or indoor gas cookers; (iii) sulphur dioxide from burning fossil fuels; and (iv) ozone at ground level, caused by the reaction of sunlight with pollutants from vehicle emissions. The pollutant that affects people the most is particulate matter (PM). This is because the microscopic particles can slip past human body's defences, penetrating deep into the respiratory and circulatory system, causing damage to lungs, heart and brain [2]. The detrimental health effects of air pollution range from acute symptoms to chronic diseases such asthma, lung cancer, cardiovascular arrests and death.

Over 80% of people living in urban areas are exposed to air quality levels that exceed WHO guideline limits, with low- and middle-income countries suffering from the highest exposures, both indoors and outdoors [1]. India is among the top countries in the world where air pollution is of particular concern. It is no surprise that 21 out of 30 world's most polluted cities are in India [3].

Monitoring and controlling the air pollution in India and across the world has therefore gained increasing importance over the past decade. The Indian government launched a National Air quality Index (AQI) in 2015 which is an alert system that notifies the public about air pollution

levels and associated health risks [4]. The Indian AQI uses bands, based on concentrations of the highest pollutant. The index uses the Indian Air Quality standards to convert concentrations into and index, and gives health advice based on the AQI band [5].

The AQI values have been categorised into 6 groups based on the level of danger that they pose to human health.

AQI	Remark	Color Code	Possible Health Impacts
0-50	Good		Minimal impact
51-100	Satisfactory		Minor breathing discomfort to sensitive people
101-200	Moderate		Breathing discomfort to the people with lungs, asthma and heart diseases
201-300	Poor		Breathing discomfort to most people on prolonged exposure
301-400	Very Poor		Respiratory illness on prolonged exposure
401-500	Severe		Affects healthy people and seriously impacts those with existing diseases

Figure 1: Indian AQI bands; Source: https://app.cpcbccr.com/AQI_India/

The air quality data is often complex and confusing as it includes information about multiple pollutants, several units and standards that require professionals to make sense of. Hence, the need to convert it into a form that is easily understood by common man. AQI is a measure of the levels of air pollution used to inform the public how polluted the air currently is or it is forecast to become. AQI information helps people make informed decisions while planning their activities based on the air quality expected on that day. Future air quality can be modelled based on current levels of the AQI so as to predict the air quality ahead of time.

This report attempts to study the **relationship between climatological factors** and **AQI_bucket**. We also aim to implement **supervised** and **unsupervised learning models** on the chosen dataset and assess their performance in predicting the AQI_bucket given the weather information of **various cities across India for 2015-2020**.

2. RELATED WORK

2.1 LITERATURE REVIEW

There has been considerable research done in the field of air pollution in the past decade. Various statistical models have been used and implemented in predicting AQI values.

In this work [6], air quality prediction was done in R language using two classification techniques, namely, Multinomial Logistic Regression and Decision Trees. Using the same data, they have also performed K-means clustering which is an unsupervised learning technique for grouping the data values into clusters of ‘good’, ‘moderate’ and ‘unhealthy’. In their evaluation of the 2 classifiers, it was found that Multinomial Logistic Regression gives a better accuracy and low error rate as compared to decision trees.

This paper [7] presents a comparative study of various regression techniques for predicting the air quality values. The classifiers include Decision tree, Random Forest, Gradient Boosting Regression and ANN Multi-layer Perceptron Regression which were evaluated across 5 different datasets based on Mean Absolute error (MAE), Root Mean Square Error (RMSE) and

processing time. It was found that Random Forest has the highest accuracy and comparatively low processing time than Gradient boosting and ANN.

A similar approach is highlighted in [8] where they have compared the strengths and limitations of various machine learning techniques on predicting air quality based on historical data. It summarizes the merits and demerits of ANN, Fuzzy Logic, Kalman Filter, Decision Tree, Random Forest used for air quality forecasting. Additionally, they also explore the tools used for real-time monitoring of air-quality based on processing of real-time data.

In this paper [9], they have used Auto-Regression (AR) and Auto-Regressive Integrated Moving Average (ARIMA) to create a model for predicting daily AQI forecast. Autoregression is a time series model that uses observations from previous time stamps as input to a regression equation to predict the value at the next time step. ARIMA is also used to predict the future vales by making them stationary through logging or deflating. [13] makes a comparison between univariate i.e. ARIMA and multivariate time series i.e. Vector Auto-Regression (VAR) models in predicting the AQI of air quality data based on Gurugram city of Haryana. The results show that ARIMA is better at AQI prediction than VAR.

[10] discusses implementing numerous supervised learning techniques such as Linear Regression, SDG Regression, Random Forest, Decision Trees, Support Vectors, ANN, Gradient Boosting and Adaptive Boosting Regression for prediction of AQI of major pollutants for data used obtained from the Central Pollution Control Board (CPCB) located in Delhi. These were evaluated based on Mean square error, Mean absolute error and R squared. The data was maintained on an IoT platform. Their findings reveal that Support Vector Regression and Artificial Neural Networks are best suited for predicting the air quality in New Delhi.

The system proposed in [11] uses real time sensors to collect the air quality data. The data is then processed using the Fuzzy c-Means clustering algorithm. The results show that Fuzzy c-Means clustering provides better results with improved accuracy and low error rate as compared to other algorithms mentioned in the literature including K-means and linear regression techniques.

A big data based platform is developed in [12] to visualise air quality data. This platform combines time and spatial dimensions through a wide range of technologies. It uses an ETL (Extract-Transform-Load) based framework which includes (i) computing and (ii) storage nodes. The primary focus of this paper is on providing a visual source of the air quality data which is easier and faster to comprehend as compared to traditional text.

This paper [14] deals with the estimation of Regression coefficient, Hurst exponent, Fractal Dimension and Predictability Index of air pollutants such as NOx, SO2, PM2.5. They have taken into account an Odd-Even scheme launched by the Indian government in an attempt to curb air pollution in the state capital, Delhi. According to this Odd-Even rationing method: cars running with number plates ending in Odd digits could only be driven on certain days of the week, while the Even digit cars could be driven on the remaining days of the week. This paper considers the weather conditions during specific time periods prior to, during, and after implementation of this scheme in the Dwarka area, adjoining the Indira Gandhi International Airport, Delhi. It was concluded that this scheme, as a standalone was not could not significantly reduce the air pollution levels especially due to the unpredictable behaviour of the air pollutants.

This paper [15] focuses on testing the usefulness of existing regression models in the sklearn library in Python to predict the AQI values given historical weather data. It also highlights the features that are most useful for the prediction. The results obtained show that the Extra Trees regression model has the highest accuracy in prediction. The dataset used in this system was based on air quality and meteorological data of New Delhi from 1-1-2015 to 24-4-2017.

This work [16] uses Binary Logistic Regression to classify air quality data sample of a specific city into 2 classes – polluted or not polluted. Furthermore, Autoregression is used to predict future levels of PM2.5 based on historical PM2.5 readings. The results show that machine learning models (logistic regression and autoregression) can be efficiently used to detect the quality of air and predict the level of PM2.5 in the future in small as well as big cities.

[17] uses various Machine Learning techniques including Linear Regression, Neural Networks (NN), and Genetic Programming for forecasting daily AQI values in Northern Thailand. They have worked on data collected during the dry season (January – May) of Thailand in the year 2018 – 2019. The results show that all three models work well for predicting air quality in an un-polluted environment reproducing an accuracy of 97%. However, things get worse when predicting AQI for unhealthy air situations. In such environments, the models built using Linear regression and Genetic Programming yield acceptable results with an accuracy of 76.67% and 80.72% respectively. In their analysis for predicting air quality in heavily polluted situations, it is found that NNs yield the lowest accuracy of 42.42%.

2.2 INTERESTING FINDINGS

Upon review of the above works, it was observed that most, if not all analyses were based on predicting **Air Quality Index (AQI)** as an **outcome variable.** The AQI, as the name suggests is a **numeric** variable which ranges from **0 to 500 and above**. However, **in this study, we aim to predict AQI_bucket which is a categorical variable.** This is because **standalone AQI values are difficult to understand for someone who does not have the necessary domain knowledge.** For instance, if the AQI value of XYZ city is shown to be 195, it is unlikely for a layman to comprehend what this figure means and the amount of health hazard it poses. Therefore, we believe analysing **AQI_bucket** which **predicts the air quality** based on categories of **'good', 'satisfactory', 'moderate', 'poor', 'very poor'** and **'severe'** would be a **better way of representing** and **communicating air quality data to the public**. The categories would send a more direct health message as compared to the numeric AQI values which are hard to interpret without any reference or background knowledge.

It is also seen that numerous machine learning models have used **Auto-Regression** for **Time-series analysis. This study does not include time series analysis.** The focus of this study is on predicting AQI_bucket based on the factors responsible for air pollution. The scope does not include predicting daily AQI values.

It is interesting to note that **Neural Networks** when used for **predicting the AQI values** perform **poorly** in comparison to other classifiers when tested **on unseen data**. This may be because they tend to **overfit the training data** hence giving a low accuracy when applied to test data. Moreover, **they are not good at handling outliers** i.e. **abrupt changes in the air quality** including changing concentrations of air pollutants. Moreover, NNs require a high processing time for bigger datasets. NNs perform well and compute faster only on small datasets.

3. DATA

The dataset used for the purpose of this analysis was obtained from the **Central Pollution Control Board: https://cpcb.nic.in/** which is the official portal of the Government of India. The data has been made publicly available.

3.1 DATASET DESCRIPTION

For the purpose of this analysis, we have focused on the dataset available from **2015-2020** which includes the **city_day.csv** file with daily values of variables.

No of variables (columns): 16

No. of observations (rows): 28,840

In this study, we have chosen **AQI_bucket** which is a categorical variable as our **outcome variable**. The **explanatory variables** that will be used for predicting the AQI_bucket include all air pollutants - **PM2.5, PM10, NO, NO2, NOx, NH3, CO, SO2, O3, Benzene, Toluene** and **Xylene**. We will not be using the variable AQI as an explanatory variable in this study.

Variable Description

- ***City***

Data Type: Factor with 25 levels
Description: Includes city names for which the air quality data is recorded in an alphabetical order.

- ***Date***

Data Type: Datetime
Description: Records the date for which the data was recorded.

- ***PM2.5***

Data Type: Numeric, Float
Description: Particulate matter of diameter less than 2.5 micrometres
Sources: power plants, motor vehicles, airplanes, residential wood burning, forest fires, agricultural burning, volcanic eruptions and dust storms

- ***PM10***

Data Type: Numeric, Float
Description: Particulate matter of diameter less than 10, micrometres, also known as respirable particulate matter.
Sources: dust from unsealed roads, smoke from fires, sea salt, car and truck exhausts, industry

- ***NO, NO2, NOx***

Data Type: Numeric, Float
Description: NO is nitric oxide, formed by oxidation of nitrogen in air whereas NO2 stands for nitrogen dioxide, a reddish-brown highly poisonous gas, formed when nitric oxide reacts

very quickly with more oxygen and NOx is nitrogen oxide, a chemical compound of oxygen and nitrogen, formed by reaction during combustion at high temperatures
Sources: Lightning stroke, volcanic eruptions, burning of fuels such as oil, diesel, gas and organic matter, emissions from cars, trucks and buses, and power plants.

- ***NH3***

Data Type: Numeric, Float
Description: NH3 (Ammonia), a colourless gas with a characteristic pungent smell, is a compound of nitrogen and hydrogen.
Sources: Agriculture-related emissions which include biomass burning or fertiliser manufacture.

- ***CO***

Data Type: Numeric, Float
Description: It is a colourless, odourless, tasteless, toxic gas, which is slightly denser than air.
Sources: Produced in the incomplete combustion of carbon-containing fuels such as gasoline, natural gas, oil, coal, and wood.

- ***SO2***

Type: Numeric, Float
Description: A toxic gas responsible for the smell of burnt matches which is released naturally by volcanic activity.
Sources: It is produced as a by-product of copper extraction and the burning of fossil fuels contaminated with sulphur compounds.

- ***O3***

Type: Numeric, Float
Description: O3 is ground-level ozone, a colorless and highly irritating gas that forms just above the earth's surface. It is called a "secondary" pollutant because it is produced when two primary pollutants react in sunlight and stagnant air.
Sources: Oil refining, printing, petrochemicals, lawn mowing, aviation, bushfires.

- ***Benzene***

Type: Numeric, Float
Description: It is a clear, colourless, highly flammable and volatile, liquid aromatic hydrocarbon with a gasoline-like odour.
Sources: Mainly found in crude oils and as a by-product of oil-refining processes.

- ***Toluene***

Type: Numeric, Float
Description: Toluene, a colourless, water-insoluble liquid, is a benzene derivative aromatic hydrocarbon with the smell associated with paint thinners
Sources: Naturally occurs in crude oil and in the process of making coke from coal, also released during forest fires.

- ***Xylene***

Type: Numeric, Float
Description: Xylene is also a benzene derivative aromatic hydrocarbon. It is a colourless liquid that has a sweet odour and is primarily a synthetic chemical.
Sources: Occurs naturally in petroleum, coal, tar, and is released during forest fires.

- ***AQI***

Type: Numeric, Float
Description: AQI stands Air Quality Index values calculated for each day

- ***AQI_Bucket***

Type: Factor with 6 levels
Description: AQI_bucket into 'Good', 'Satisfactory', 'Moderate', 'Poor', 'Very Poor', 'Severe' describing various health hazards.

Measures of Central Tendency

As part of data visualisation and to understand the spread of the data points for all our variables, we have calculated the measures of central tendency for our numeric variables. As AQI_bucket, city and date are non-numeric/categorical variables, measures of central of central tendency are meaningless.

	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	Benzene	Toluene	Xylene	AQI
count	24250.000000	17795.000000	25313.000000	25266.000000	24659.000000	18635.000000	26784.000000	24989.000000	24821.000000	23220.000000	20802.000000	11300.000000	24201.000000
mean	68.508440	120.465929	17.848399	28.722892	32.538442	23.733286	2.285140	14.716054	34.498114	3.324055	8.903265	3.093919	168.846453
std	65.207394	90.977792	22.970994	24.621160	31.947611	25.948930	7.047618	18.341909	21.829299	16.013089	20.245367	6.352506	141.737744
min	0.040000	0.010000	0.020000	0.010000	0.000000	0.010000	0.000000	0.010000	0.010000	0.000000	0.000000	0.000000	13.000000
25%	29.472500	58.925000	5.710000	11.820000	12.710000	8.700000	0.510000	5.660000	18.770000	0.140000	0.700000	0.140000	83.000000
50%	49.620000	97.880000	10.080000	21.880000	23.730000	16.040000	0.900000	9.240000	30.830000	1.110000	3.090000	0.995000	120.000000
75%	81.940000	152.175000	20.380000	37.820000	40.480000	30.210000	1.460000	15.510000	45.580000	3.142500	9.330000	3.390000	212.000000
max	949.990000	1000.000000	390.680000	362.210000	467.630000	352.890000	175.810000	193.860000	257.730000	455.030000	454.850000	170.370000	2049.000000

Figure 2: Measures of central tendency for numeric variables

3.2 DATA PRE-PROCESSING

Since the data was clean as it is, cleaning was not required. However, it was observed that we have the following missing values in our data:

	Total Values	Missing Values	% of Total Values
Xylene	11300	17540	60.800000
PM10	17795	11045	38.300000
NH3	18635	10205	35.400000
Toluene	20802	8038	27.900000
Benzene	23220	5620	19.500000
AQI	24201	4639	16.100000
AQI_Bucket	24201	4639	16.100000
PM2.5	24250	4590	15.900000
NOx	24659	4181	14.500000
O3	24821	4019	13.900000
SO2	24989	3851	13.400000
NO2	25266	3574	12.400000
NO	25313	3527	12.200000
CO	26784	2056	7.100000

Figure 3: Missing values in dataset

Due to a large number of missing values in our dataset, we cannot remove them as doing so might result in loss of crucial information. Thus, we need to handle them. There are multiple ways to handling missing values which replacing them with:

- A constant value that has meaning within the domain, such as 0, distinct from all other values
- A value from another randomly selected record
- A mean, median or mode value for the column
- A value estimated by another predictive model

Upon checking for the kurtosis and skewness of our variables/columns, it was observed that all the variables have a **positive kurtosis** and are **rightly skewed**. Therefore, using mean, median or mode to replace the missing values is not a good idea as they might not be representative of the sample and overlook outliers in the dataset.

```
PM2.5        20.767930
PM10          6.672594
NO           24.757759
NO2          11.165340
NOx          10.613212
NH3          27.465288
CO          106.746701
SO2          21.444798
O3            3.421709
Benzene     518.564080
Toluene     211.415421
Xylene      118.916184
AQI          21.115624
dtype: float64
```

Figure 4: Kurtosis values

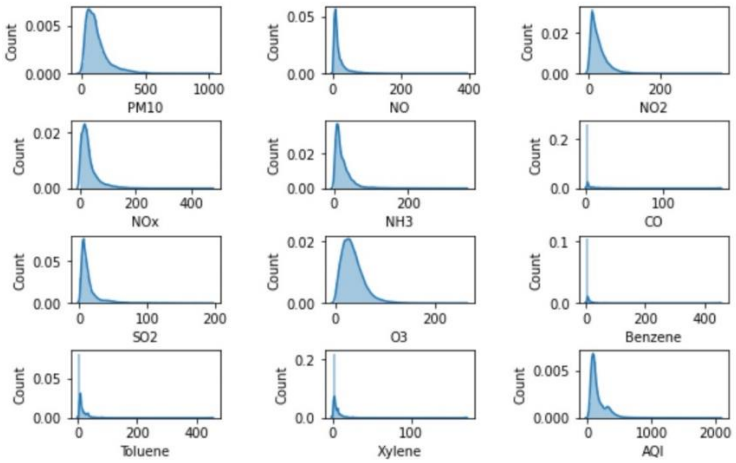


Figure 5: Skewness plots

Next, we tried imputing using predictive models, namely, **KNN Imputer** and **Multiple Imputation by Chained Equations (MICE)**. KNN uses 'feature similarity' to impute the missing values whereas MICE performs multiple regressions on the data to get a better idea of the uncertainty in the missing values and then replaces those. Upon trying both the methods, we obtained an accuracy of **56.16%** for **KNN Imputer** and **60.43%** for **MICE**. **Thus, we decided to proceed with MICE for replacing the numeric missing values in our dataset.**

As for replacing the missing values of our outcome variable **"AQI_bucket"** which is a categorical variable, we used domain knowledge to fill those up. AQI values were used to fill the AQI_bucket missing values.

4. ANALYSIS

4.1 FEATURE ENGINEERING

Feature engineering is the process of using domain knowledge to extract features from raw data by means of data mining techniques. The extracted features can then be used to improve the performance of machine learning algorithms.

We begin with **all air pollutants** as **explanatory variables** and **AQI_bucket** as our **outcome variable** and create a **baseline model** using **Multinomial Logistic Regression as the classifier**. The **train/test split ratio** used for the dataset is **80/20**. At this point, we have not selected any features. With this model, we get an **accuracy** of **59.65%**. The baseline model does not give us any important features.

Next, we use **Recursive Feature Elimination (RFE)** to perform **feature selection**. RFE is a feature extraction method that fits a classification model and removes the weakest feature/s in every iteration until the specified number of features is reached. However, we do not know what is a good number of features that should be kept in our model. Therefore, to reach this optimal number of features, we use **cross-validation** in combination **with RFE** to rank different subsets of features. Ultimately, the best scoring subset of features is used for building our model. The **Recursive Feature Elimination, Cross-Validated (RFECV) visualizer** has been used to plot the cross-validation score relative to each subset and see the trend in feature elimination. It can be observed that the score jumps from low to high as the number of features removed increases, peaks at 9 features and then slowly decreases again.

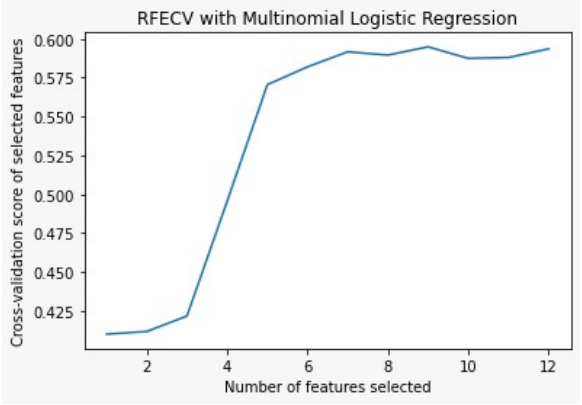


Figure 6: RFECV visualisation plot; peak accuracy: 60.19%

For this method, we obtained an **accuracy** of **60.19%** with **9 significant features,** namely, **PM2.5, NO, NOx, NH3, CO, O3, Benzene, Toluene, Xylene.** This accuracy is fairly better as compared to the baseline model.

As we have now narrowed down on features which are important, going forward, we will be using these 9 features to fit the model for the rest of our classifiers.

4.2 SUPERVISED LEARNING MODELS

Supervised learning is the machine learning task of learning a function that maps an input to an output based on example input-output pairs. it infers a function from labeled training data consisting of a set of training examples.

Experiment 1: To compare the performance of different supervised learning techniques in predicting the AQI_bucket based on the levels of air pollutants for a **train/test data split of 80/20**.

The following table summarizes the results obtained for the above data split across various performance metrics:

Parameter/Models	Multinomial Logistic Regression	Naïve Bayes	Support Vector	K-Nearest Neighbour	Random Forest
Accuracy	60.19	67.83	75.17	77.42	81.10
F1 score	57.10	67.35	73.47	77.18	80.96
Precision	56.31	67.72	74.68	77.32	81.00
Recall	60.19	67.83	75.17	77.42	81.10

It can be observed that **Random Forest classifier** performs the **best** among all other classifiers followed by KNN, SVM, Naïve Bayes and lastly Multinomial Logistic Regression across all metrics. However, we cannot be sure if the above classifiers are reliable as the accuracy obtained for one test set can be very different from the accuracy obtained for a different test set. **K-fold Cross Validation(CV)** provides a solution to this problem as it works by **dividing the data** into **k folds** and ensuring that **each fold** is used as a **testing set** at **some points.** Therefore, in the following experiment, we make use of K-fold cross validation to check if a similar trend is observed in the classifiers.

Experiment 2: To compare the performance of different supervised learning techniques in predicting the AQI_bucket based on the levels of air pollutants using **K-fold cross validation**.

The following table shows the accuracy obtained for the classifiers upon using K-fold cross validation.

Parameter/Models	Multinomial Logistic Regression	Naïve Bayes	Support Vector	K-Nearest Neighbour	Random Forest
Accuracy	55.85	69.44	75.03	70.81	75.15

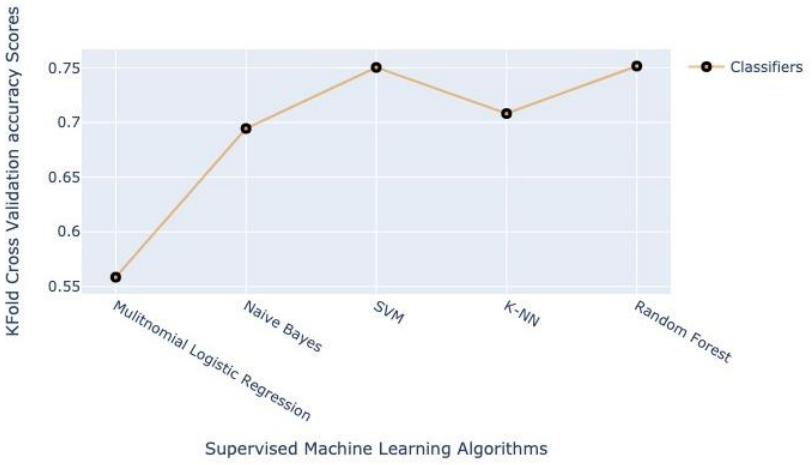


Figure 7: Accuracy scores for classifiers using K-fold cross-validation

Based on the above results, it can be observed that **overall accuracy for all classifiers reduces** with the **exception of Naïve Bayes**. However, **Random Forest** is **still the best classifier** for predicting AQI_bucket. **SVM** also has a comparable accuracy value close to 75% which is the same as what we obtained in the 80/20 data splitting. This means **SVM is robust** to **unseen data.** However, the **accuracy of KNN classifier reduces significantly** for k-fold cross-validation. This implies that it does not generalize well on unseen data.

4.3 UNSUPERVISED LEARNING

Unsupervised learning is a machine learning task that **draws inferences from datasets** consisting of **input data** without **labelled class values**. The most common unsupervised learning methods include **Cluster analysis** and **Association Rule Mining** which can be used to find hidden patterns or grouping in data.

It is **not feasible** to implement **Association Rule Mining algorithm** on the dataset chosen due to the **large dataset size** (28,840 observations) and **9 numeric feature variables** (all explanatory variables are numeric). Therefore, we will use **K-Means clustering** for our analysis. K-means clustering **works best** on **numeric data**. It forms groups/clusters based on distance metrics such as Euclidean/Manhattan distance. Once the algorithm has run and groups are defined, any new data can be assigned to the relevant group.

We first need to choose the number of clusters, k. For choosing the optimum value of k, we use **Elbow method** and **Silhouette method**. The Elbow method uses Within-Cluster-Sum of Squared Errors (WSS) for different values of k, and choose the k for which WSS becomes first starts to diminish. In the plot of WSS-versus-k, this is visible as an elbow. The following elbow plot has a visible kink at k = 3 clusters.

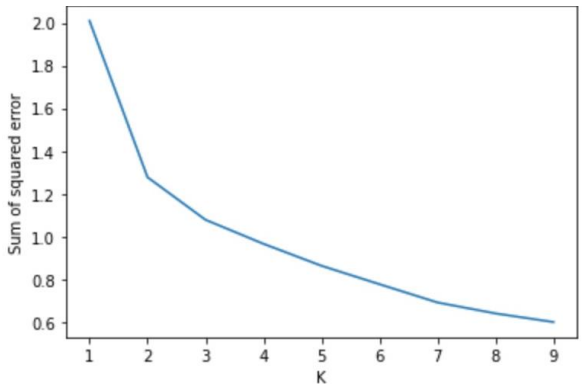


Figure 8: Elbow plot

The Silhoutte method measures similarity of an object with its own cluster (cohesion) as compared to other clusters (separation). It gives accuracy values ranging from −1 to +1, where a high value indicates that the object is well matched to its own cluster and poorly matched to neighbouring clusters.

```
For n_clusters =  2 average silhouette score: 0.5885621910785082
For n_clusters =  3 average silhouette score: 0.42892034332382617
For n_clusters =  4 average silhouette score: 0.2480606733802525
For n_clusters =  5 average silhouette score: 0.2661134041176675
For n_clusters =  6 average silhouette score: 0.27028447631211877
For n_clusters =  7 average silhouette score: 0.2774272190446317
For n_clusters =  8 average silhouette score: 0.2750010864319877
For n_clusters =  9 average silhouette score: 0.2623702065106464
```

Figure 9: Silhoutte method

It is observed that the accuracy decreases significantly after 3 clusters from 0.42 to 0.24. Thus, n_clusters = 3 is used for further analysis.

We now plot the 3 clusters for pollutants PM2.5 vs. NO and PM2.5 vs. O3 to see the spread of data points. The following graph shows the 3 clusters in yellow, blue and pink. To make sense of these clusters, let us take into consideration three categories for air quality: Good, Moderate and Bad and consider the following clusters.

- Cluster 1 = yellow
- Cluster 2 = blue
- Cluster 3 = pink

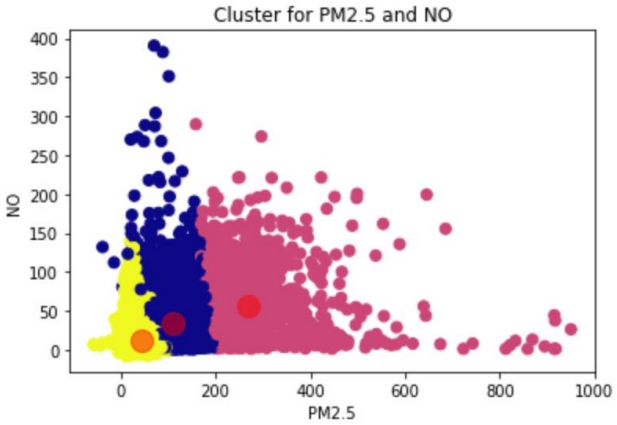


Figure 10: PM2.5 and NO concentrations

In order to make sense of the clusters, we used domain knowledge of the field. In India, major metropolitan cities have ‘Severe’ AQI values. Moreover, the Indian Central pollution control board has set up industrial and residential standards for all the pollutants concentrations with respect to the AQI for different cities in India. Using the standards, it can be interpreted that at **low levels of PM2.5 and NO**, the air quality is likely to be **good**. Therefore, the **yellow cluster** corresponds to **“Good” air quality**. At **moderate concentrations** of PM2.5 and a mix of high and low NO levels, the air quality can be interpreted to be “**Moderate”**. Thus, the blue cluster corresponds to moderate air quality. At extremely **high concentrations** of PM2.5, knowing that PM2.5 is the major pollutant responsible for poor air quality, the pink cluster is likely to have **“Poor”** air quality.

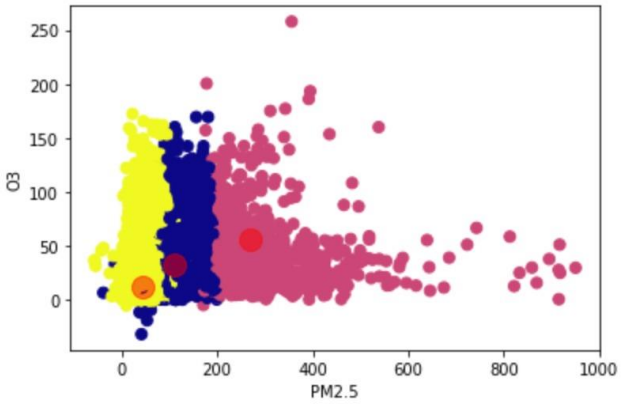


Figure 11: PM2.5 and O3 concentrations

A similar logic trend can be used to interpret the 3 clusters corresponding to the above given air quality categories. Thus, clustering be used to infer groups of air quality data based on varying concentrations of particulate matter, NO and O3.

5. IMPACT OF COVID19 – INTERESTING FINDINGS

In our analysis, we found some interesting patterns/trends in the data. In response to the global pandemic of COVID-19 (an infectious disease caused by the coronavirus which leads to serious respiratory diseases), a nationwide lockdown was imposed in India. Initially for three weeks from 24th March to 14th April 2020, it extended up to 3rd May 2020 and then to June end where after the restrictions were eased out slowly in phases.

Due to the forced restrictions in place, pollution level in cities across Indian cities drastically reduced just within few days [18]. This is evident from the graphs that show the striking difference in AQI levels starting April 2020.

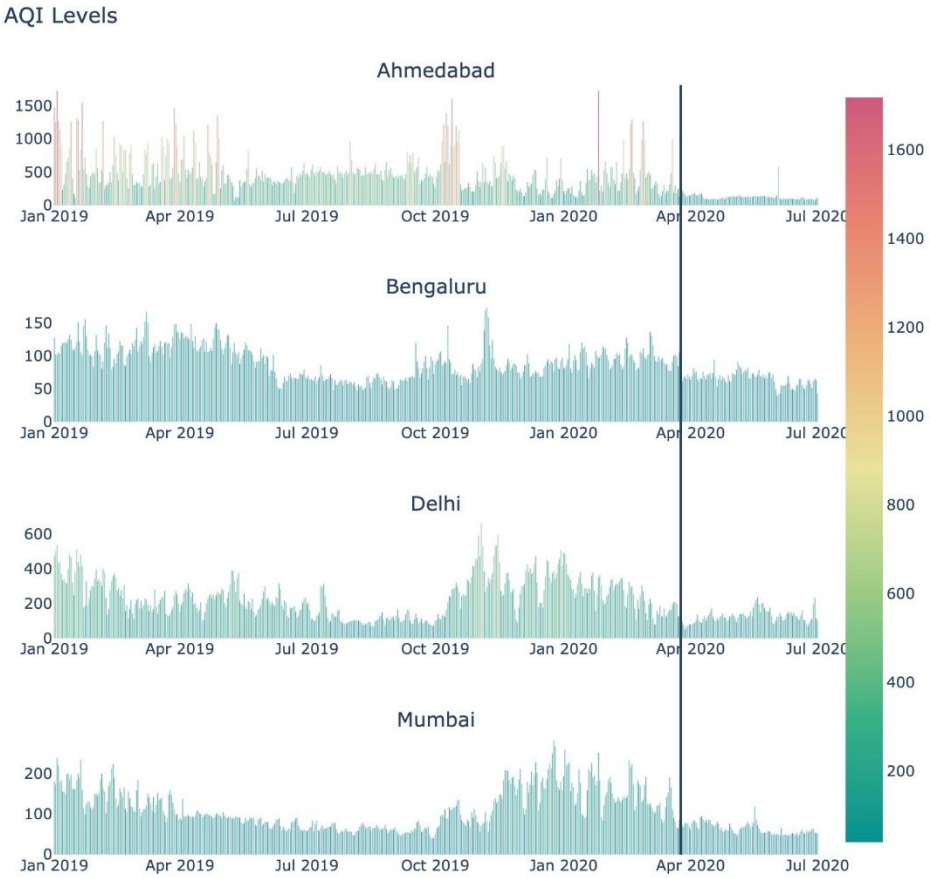


Figure 12: AQI levels across 4 metropolitan cities of India

The black line drawn across the 4 metropolitan cities of Ahmedabad, Bengaluru, Delhi and Mumbai shows reduced AQI levels indicating reduced air pollution due to stringent lockdown. This is primarily because the nationwide lockdown resulted in a drastic decline in NO2 emissions from human sources such as road transport, planes, power plants and ships, all of which burn fossil fuels which are major sources of NO2. This decline is evident from the graph below which shows NO2 concentrations for major Indian cities (Ahmedabad, Bengaluru, Delhi, Mumbai) across 2019 and 2020.

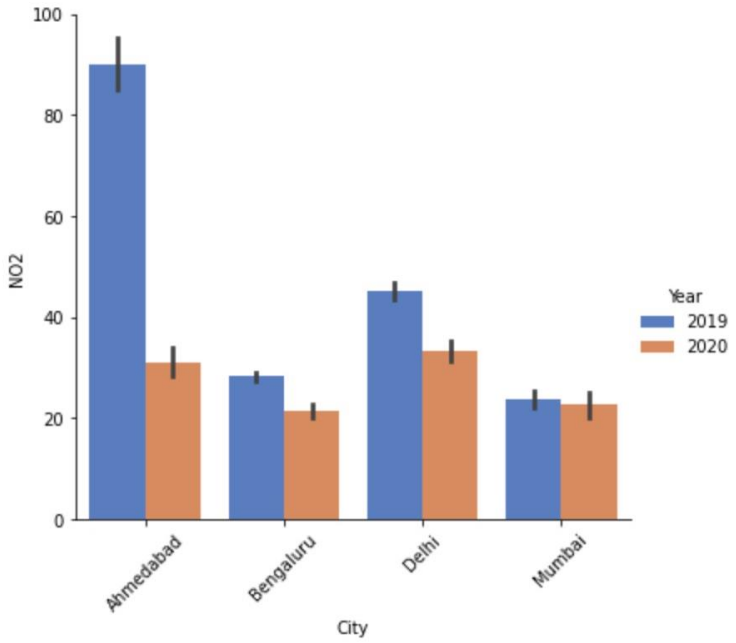


Figure 13: NO2 levels for major cities for years 2019 & 2020

The graph below shows the change in concentrations of NO2, PM 2.5, NH3, O3 and NH3 over the period of 2015-2020. It can be seen that levels of these gaseous pollutants have reduced drastically in 2020.

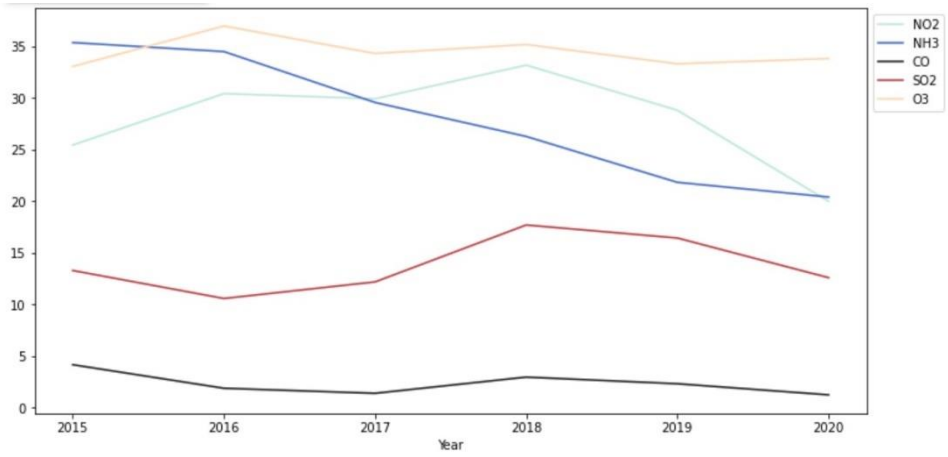


Figure 14: Concentrations of air pollutants over the years

Such an effect has spurred discussions about resorting to lockdown as an effective alternative for controlling air pollution. The only way forward is to maintain this effect even after the lockdown ends.

6. CONCLUSION

Since air pollution is a major environmental hazard, it cannot be neglected especially now than ever before. Thus, it is important to monitor air quality and implement regulatory measures to curb air pollution for a better future.

Prior works have addressed this issue however, their research was based on predicting the numeric AQI values and time series analysis. In our analysis, we have implemented 5 classifiers (Multinomial Logistic Regression, Naive Bayes, KNN, Random Forest and SVM) for predicting AQI_bucket. These classifiers were modelled using 9 features selected through the implementation of Recursive Feature Elimination with cross validation technique. The results show that Random forest and SVM classifiers have the highest prediction accuracy using K-fold cross-validation. Additionally, the use of K-means clustering produced 3 meaningful clusters based on concentrations of PM2.5, NO2 and O3. In order to make sense of these clusters, domain knowledge is very crucial.

7. FUTURE SCOPE

Monitoring air quality data alone is not enough to curb air pollution. In order bring about the desired change in air quality levels, people must change their behaviours and actions. Furthermore, to make the current drops in air pollution levels permanent, there is a need for radical policies/laws regarding transportation, fuel consumption etc. Analysing the effect of lockdown during Covid19, it can observed that low levels of air pollution levels can be correlated to reduction in road, rail and air transport. Moreover, it can be inferred that fuel-powered cars are key drivers of air pollution.

According to the European Environment Agency (EEA) report, battery electric cars emit less greenhouse gases and air pollutants over their entire life cycle than petrol and diesel cars [19]. It is suggested that renewable energy and circular economy — including the shared use of vehicles and product design that supports reuse and recycling — will help maximise the benefits of shifting to electric vehicles.

Thus, in order to curb the effects of exacerbating air pollution, Indians must consider switching to electric cars. On the other hand, the government must look into developing faster public transportation, constructing bike lanes, and finding other ways to incentivise people such as rebates and tax reductions (as done in Canada for encouraging use of Electric vehicles) to help bring about this change. Such a shift would dramatically reduce India’s emissions from its primary human source of air pollution. It’s also important that these electric vehicles, and India’s cities are powered by clean sources of energy rather than fossil fuels.

REFERENCES

1. [Online]https://www.who.int/health-topics/air-pollution#tab=tab_1
2. [Online]https://www.who.int/airpollution/news-and-events/how-air-pollution-is-destroying-our-health
3. [Online] https://www.cnn.com/2020/02/25/health/most-polluted-cities-india-pakistan-intl-hnk/index.html
4. [Online]https://www.nrdc.org/experts/anjali-jaiswal/india-announces-national-air-quality-index-increase-public-awareness
5. Jason Thongplang, *"What is an air quality index and what are the benefits?"*, Apr 1, 2016, aeroqual.com
6. K. Nandini; G. Fathima, *"Urban Air Quality Analysis and Prediction Using Machine Learning"*, 2019 IEEE, 978-1-7281-0418-8/19/©2019 IEEE
7. Saba Ameer, Munam Ali Shah, Abid Khan, Houbing Song, Carsten Maple, Saif ul Islam, Muhammad Nabeel Asghar, *"Comparative analysis of machine learning techniques for predicting air quality in smart cities"*, DOI 10.1109/ACCESS.2019.2925082, IEEE Access 2017
8. Varsha Khandekar, Dr. Pravin Srinath, *"Machine Learning Techniques for Air Quality Forecasting and Study on Real-Time Air Quality Monitoring"*, Third International Conference on Computing, Communication, Control And Automation (ICCUBEA), 978-1-5386-4008-1/17/©2017 IEEE
9. Nimisha Tomar, Durga Patel, Akshat Jain, *"Air Quality Index Forecasting using Auto-regression Models"*, IEEE International Students' Conference on Electrical, Electronics and Computer Science, 2020
10. Chavi Srivastava, Shyamli Singh, Amit Prakash Singh, *"Estimation of Air Pollution in Delhi Using Machine Learning Techniques"*, pg 304-309 International Conference on Computing, Power and Communication Technologies (GUCON), 978-1-5386-4491-1/18/ ©2018 IEEE
11. R. Kingsy Grace, Karthika Aishvarya.S, Monisha.B, Kaarthik.A, *"Analysis and Visualization of Air Quality Using Real Time Pollutant Data"*, pg 34-39 6th International Conference on Advanced Computing & Communication Systems (ICACCS), 978-1-7281-5197-7/20/ ©2020 IEEE
12. Yu-Ren Zeng , Yue Shan Chang , You Hao Fang , *"Data Visualization for Air Quality Analysis on Bigdata Platform"*, pg 313-317 International Conference on System Science and Engineering (ICSSE), 978-1-7281-0525-3/19/ ©2019 IEEE
13. Jasleen Kaur Sethi, Mamta Mittal, *"Analysis of Air Quality using Univariate and Multivariate Time Series Models"*, pg 823-10th International Conference on Cloud Computing, Data Science & Engineering (Confluence), 978-1-7281-2791-0/20/ ©2020 IEEE
14. Rashmi Bhardwaj, Dimple Pruthi, *"Time series and Predictability analysis of Air Pollutants in Delhi"*, 2nd International Conference on Next Generation Computing Technologies (NGCT-2016) Dehradun, India 14-16 October 2016
15. Soubhik Mahanta, T. Ramakrishnudu, Rajat Raj Jha, Niraj Tailor, *"Urban Air Quality Prediction Using Regression Analysis"*, pg 1118-1123 IEEE Region 10 Conference (TENCON 2019), 978-1-7281-1895-6/19/©2019 IEEE
16. Aditya C R, Chandana R Deshmukh, Nayana D K, Praveen Gandhi Vidyavastu, *"Detection and Prediction of Air Pollution using Machine Learning Models"*, pg 204-207 International Journal of Engineering Trends and Technology (IJETT) – volume 59 Issue 4 – May 2018
17. Supawadee Srikamdee, Janya Onpans,*"Forecasting Daily Air Quality in Northern Thailand Using Machine Learning Techniques"*, 24th International Conference on Information Technology (InCIT), Bangkok, Thailand, 978-1-7281-1019-6/19/ ©2019 IEEE
18. Manuel A. Zambrano-Monserrate, María Alejandra Ruano, Luis Sanchez-Alcalde, *"Indirect effects of COVID-19 on the environment"*, Science of the Total Environment, 2020
19. [Online] https://www.eea.europa.eu/highlights/eea-report-confirms-electric-cars

APPENDIX

Importing Libraries

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.impute import SimpleImputer
from sklearn.impute import KNNImputer
from fancyimpute import IterativeImputer

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.feature_selection import RFE
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.cluster import KMeans

from sklearn import metrics
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score
from sklearn.metrics import classification_report
from sklearn.metrics import silhouette_score

import warnings
warnings.filterwarnings('ignore')

import itertools
%matplotlib inline
```

Importing Csv File

```
from google.colab import files
data_to_load = files.upload()

import io
```

```
main = pd.read_csv(io.BytesIO(data_to_load['city_day.csv']))
main.head()
m1=main.copy()
m1.info()

Correlation of numeric variables with AQI
corr_with_AQI = m1.corr().AQI.sort_values(ascending = False)
corr_with_AQI
```

Checking the data distribution for all the numeric variables (Skewness and Kurtosis)

```
columns = m1.columns[3:15]
plt.subplots(figsize=(9,9))
length =len(columns)
for i,j in itertools.zip_longest(columns,range(length)):
    plt.subplot((length/2),3,j+1)
    plt.subplots_adjust(wspace=0.7,hspace=0.7)
    plt.ylabel("Count")
    sns.distplot(m1[i].dropna())
    #plt.title(i)
plt.show()

m1.kurtosis()
```

Handling Missing Values

```
#Checking Missing values
def missing_values_table(m1):

        #Total values
        val =m1.count()
        # Total missing values
        mis = m1.isnull().sum()

        # Percentage of missing values
        mis_percent = 100 * m1.isnull().sum() / len(m1)

        # Make a table with the results
        mis_table = pd.concat([mis,val, mis_percent], axis=1)

        # Rename the columns
        mis_table_columns = mis_table.rename(
        columns = { 0: 'Missing Values', 1 : 'Existing Values', 2 : '% of Total Values'})
```

```
        # Sort the table by percentage of missing descending
        mis_table_columns = mis_table_columns[
            mis_table_columns.iloc[:,1] != 0].sort_values(
        '% of Total Values', ascending=False).round(1)

        # Print some summary information
        print ("Your selected dataframe has " + str(df.shape[1]) + " columns.\n"
            "There are " + str(mis_table_columns.shape[0]) +
              " columns that have missing values.")

        # Return the dataframe with missing information
        return mis_table_columns

missing_values= missing_values_table(m1)
missing_values.style.background_gradient(cmap='Blues')
```

Using MICE Imputation to handle missing values

```
#Using MICE Imputer to handle missing values

mice_imputer = IterativeImputer()

imputing_cols = [ 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2','O3', 'Benzene', 'Toluene',
'Xylene', 'AQI']
# we eliminated city, date, Year_Month and AQI_Bucket because
# they either were unique or had numerical substitude in other fields(AQI_bucket)
m2 = m1.copy()
mice_imputer.fit(m2[imputing_cols])

imputed = mice_imputer.transform(m2[imputing_cols])

m2.loc[:, imputing_cols] = imputed

#Handling missing value for the categorical variable "AQI_Bucket"

def get_remark(aqi_val):
    if 0<=aqi_val<=50:
        return 'Good'
    if 51<=aqi_val<=100:
        return 'Satisfactory'
    if 101<=aqi_val<=200:
        return 'Moderate'
```

```
    if 201<=aqi_val<=300:
        return 'Poor'
    if 301<=aqi_val<=400:
        return 'Very Poor'
    else:
        return 'Severe'
m2['AQI_Bucket'] = m2['AQI'].map(get_remark)
```

Feature Engineering

```
#Spliting the data
df = m2.copy()

x = df.drop(['City', 'Date', 'AQI', 'Year', 'AQI_Bucket', 'Month'], axis = 1)
y = df.AQI_Bucket

x1 = x.copy()
x1_train, x1_test, y1_train, y1_test =  train_test_split(x1,y,test_size=0.2, random_state = 27)

#Building a performance Measure function
def generate_perform_measures(model, x, y):
    ac = accuracy_score(y,model.predict(x))
    f_score = f1_score(y,model.predict(x), average='weighted' )
    prec = precision_score(y,model.predict(x), average='weighted')
    re = recall_score(y,model.predict(x), average='weighted')
    print('Accuracy is: ', ac)
    print('F1 score is: ', f_score)
    print('Precision is: ', prec)
    print('Recall is: ', re)
    return

#Baseline Model used for the comparison of the RFECV results
#Baseline Logistic Regression Model
rfe_base = LogisticRegression(multi_class= 'multinomial')

base_model = rfe_base.fit(x1_train,y1_train)

base_model.score(x1_test, y1_test)

#Recursive Feature Engineering
from sklearn.feature_selection import RFE
rfe = RFE(estimator=rfe_base, step=1)
```

```
rfe = rfe.fit(x1_train, y1_train)

selected_rfe_features = pd.DataFrame({'Feature':list(x1_train.columns),
                                      'Ranking':rfe.ranking_})
selected_rfe_features.sort_values(by='Ranking')

x1_train_rfe = rfecv.transform(x1_train)
x1_test_rfe = rfecv.transform(x1_test)


#Model for RFE
rfe_log = LogisticRegression(multi_class= 'multinomial')
rfe_log.fit(x1_train_rfe, y1_train)

rfe_log.score(x1_test_rfe, y1_test)

generate_perform_measures(log, x1_test_rfe, y1_test)

#Recursive Feature engineering with cross validation
from sklearn.feature_selection import RFECV
rfecv = RFECV(estimator= rfe_base, step=1, cv=5, scoring='accuracy')
rfecv = rfecv.fit(x1_train, y1_train)

print('Optimal number of features :', rfecv.n_features_)
print('Best features :', x1_train.columns[rfecv.support_])

rfecv.grid_scores_

plt.figure()
plt.xlabel("Number of features selected")
plt.ylabel("Cross-validation score of selected features")
plt.title("RFECV with Multinomial Logistic Regression")
plt.plot(range(1, len(rfecv.grid_scores_) + 1), rfecv.grid_scores_)
plt.show()

#Selecting the features

x1_train_rfecv = rfecv.transform(x1_train)
x1_test_rfecv = rfecv.transform(x1_test)
```

SUPERVISED LEARNING

1. Multinomial Logistic Regression

```
#Log regression
log = LogisticRegression(multi_class= 'multinomial')
log_model = log.fit(x1_train_rfecv, y1_train)

log_model.score(x1_test_rfecv, y1_test)

generate_perform_measures(log, x1_test_rfecv, y1_test)



#K fold cross validation for Multinomial Log Regression

from sklearn.model_selection import cross_val_score
acc_log = cross_val_score(estimator = log, X = x1, y=y, cv=10)
print("Average Accuracies: ",np.mean(acc_log))
print("Standart Deviation Accuracies: ",np.std(acc_log))

#Confusion Matrix for Multinomial Logistic Regression

y_pred1= log_model.predict(x1_test_rfecv)
y_act1 = y1_test
labels = ['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']
cm = confusion_matrix(y_act1, y_pred1, labels, normalize='true')
ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells

# labels, title and ticks
ax.set_xlabel('Predicted labels');ax.set_ylabel('Actual labels');
ax.set_title('Confusion Matrix For Multinomial Logistic Regression');
ax.xaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']);
ax.yaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe'])
```

2. Naive Bayes Classification

```
#Naïve Bayes
naivebayes = GaussianNB()

naive = naivebayes.fit(x1_train_rfecv, y1_train)

naive_score = naivebayes.score(x1_test_rfecv,y1_test)
```

```
naive_score

generate_perform_measures(naive, x1_test_rfecv, y1_test)

#K fold cross validation for Naïve Bayes

from sklearn.model_selection import cross_val_score

acc_naive = cross_val_score(estimator = naive, X = x1, y=y, cv=10)
print("Average Accuracies: ",np.mean(acc_naive))
print("Standart Deviation Accuracies: ",np.std(acc_naive))

#Confusion Matrix for Naïve Bayes

y_pred1= naive.predict(x1_test_rfecv)
y_actual1 = y1_test
labels = ['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']
cm = confusion_matrix(y_act1, y_pred1, labels, normalize='true')
ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax);

# labels, title and ticks
ax.set_xlabel('Predicted labels');ax.set_ylabel('Actual labels');
ax.set_title('Confusion Matrix For NAIVE BAYES');
ax.xaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']);
ax.yaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe'])
```

3. SVM Classifier

```
#SVM

svm = SVC(random_state=42)
svm_model = svm.fit(x1_train_rfecv,y1_train)
generate_perform_measures(svm, x1_test_rfecv, y1_test)

#K fold cross validation for SVM
from sklearn.model_selection import cross_val_score

acc_svm = cross_val_score(estimator = svm, X = x1, y=y, cv=10)
print("Average Accuracies: ",np.mean(acc_svm))
print("Standart Deviation Accuracies: ",np.std(acc_svm))

#Confusion Matrix for SVM
```

```
y_pred1= svm.predict(x1_test_rfecv)
y_actual1 = y1_test
labels = ['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']
cm = confusion_matrix(y_act1, y_pred1, labels, normalize='true')

ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells

# labels, title and ticks
ax.set_xlabel('Predicted labels');ax.set_ylabel('Actual labels');
ax.set_title('Confusion Matrix For SVM Classifier');
ax.xaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']);
ax.yaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe'])
```

4. KNN Classifier

```
#Optimal K value for KNN Classifier

score = []
for each in range(1,100):
    knn_find = KNeighborsClassifier(n_neighbors = each)
    knn_find.fit(x1_train_rfecv,y1_train)
    score.append(knn_find.score(x1_test_rfecv,y1_test))

plt.figure(1, figsize=(10, 5))
plt.plot(range(1,100),score,color="black",linewidth=2)
plt.title("Optimum K Value")
plt.xlabel("K Values")
plt.ylabel("Score(Accuracy)")
plt.grid(True)
plt.show()

#KNN

knn = KNeighborsClassifier(n_neighbors = 15)
knn.fit(x1_train_rfecv,y1_train)
generate_perform_measures(knn, x1_test_rfecv, y1_test)

#K fold cross validation for KNN

from sklearn.model_selection import cross_val_score
```

```
acc_knn = cross_val_score(estimator = knn, X = x1, y=y, cv=10)
print("Average Accuracies: ",np.mean(acc_knn))
print("Standart Deviation Accuracies: ",np.std(acc_knn))

#Confusion Matrix for KNN

y_pred1= knn.predict(x1_test_rfecv)
y_actual1 = y1_test
labels = ['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']
cm = confusion_matrix(y_act1, y_pred1, labels, normalize='true')
ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells

# labels, title and ticks
ax.set_xlabel('Predicted labels');ax.set_ylabel('Actual labels');
ax.set_title('Confusion Matrix For KNN Classifier');
ax.xaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']);
ax.yaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe'])
```

5. Random Forest Classifier

```
#Random Forest

random = RandomForestClassifier()
random.fit(x1_train_rfecv, y1_train)
generate_perform_measures(random, x1_test_rfecv, y1_test)

#K fold cross validation for Random Forest Classifier
from sklearn.model_selection import cross_val_score

acc_ran = cross_val_score(estimator = random, X = x1, y=y, cv=10)
print("Average Accuracies: ",np.mean(acc_ran))
print("Standart Deviation Accuracies: ",np.std(acc_ran ))

#Confusion Matrix for Random Forest

y_pred1= random.predict(x1_test_rfecv)
y_actual1 = y1_test
labels = ['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']
cm = confusion_matrix(y_act1, y_pred1, labels, normalize='true')
ax= plt.subplot()
sns.heatmap(cm, annot=True, ax = ax); #annot=True to annotate cells
```

```
# labels, title and ticks
ax.set_xlabel('Predicted labels');ax.set_ylabel('Actual labels');
ax.set_title('Confusion Matrix For Random Forest Classifier');
ax.xaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe']);
ax.yaxis.set_ticklabels(['Good', 'Satisfactory', 'Moderate','Poor','Very Poor','Severe'])

#Visualizing accuracy score for different Supervised Learning Methods
import plotly.graph_objs as go
from plotly.offline import init_notebook_mode, iplot
Kfold_acc = [np.mean(acc_log),np.mean(acc_naive),np.mean(acc_svm), np.mean(acc_knn),
np.mean(acc_ran)]

Algo_name=["Mulitnomial Logistic Regression","Naive Bayes","SVM", "K-NN","Random Forest"]
trace1 = go.Scatter(
   x = Algo_name,
   y = Kfold_acc,
   name = "Classifiers",
   marker = dict(color ='rgba(227,126,0,0.5)',
            line = dict(color='rgb(0,0,0)', width=3)),
   showlegend = True,
   cliponaxis = True,
)
go.Scatter()
data = [trace1]

layout = go.Layout(barmode = "group",
              xaxis= dict(title= 'Supervised Machine Learning Algorithms', ticklen= 5,zeroline= False),
           yaxis = dict(title= 'KFold Cross Validation accuracy Scores',ticklen= 5,zeroline= False))
fig = go.Figure(data = data, layout = layout)
iplot(fig)
```

UNSUPERVISED LEARNING

Clustering

```
#Silhouette Score
c1 = m2.copy()
m3 = c1.drop(['AQI_Bucket','City','Date','PM10','NO2','SO2', 'Year','Month','AQI'], axis =1)
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
matrix = pd.DataFrame(m3)
for n_clusters in range(2,10):
  kmeans = KMeans(init ='k-means++', n_clusters= n_clusters, n_init=100)
```

```
  kmeans.fit(matrix)
  clusters = kmeans.predict(matrix)
  silhouette_avg = silhouette_score(matrix, clusters)
  print("For n_clusters = ", n_clusters, "average silhouette score:", silhouette_avg)

#Elbow Method
sse = []
k_rng = range(1,10)
for k in k_rng:
    km = KMeans(n_clusters=k)
    km.fit(matrix)
    sse.append(km.inertia_)

plt.xlabel('K')
plt.ylabel('Sum of squared error')
plt.plot(k_rng,sse)

#K Means Clustering
n_clusters = 3

kmeans = KMeans(init = 'k-means++', n_clusters = n_clusters, n_init = 30)
kmeans.fit(matrix)
answer = kmeans.predict(matrix)
matrix ['cluster']= answer
matrix.head()
matrix.info()

print(kmeans.cluster_centers_)

#Visualizing the Clusters
fig
plt.scatter(matrix['PM2.5'], matrix['NO'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('PM2.5')
plt.ylabel('NO')
plt.title('Cluster for PM2.5 and NO')

plt.scatter(matrix['PM2.5'], matrix['O3'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('PM2.5')
plt.ylabel('O3')
```

```
plt.scatter(matrix['PM2.5'], matrix['CO'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('PM2.5')
plt.ylabel('CO')

plt.scatter(matrix['CO'], matrix['NO'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('CO')
plt.ylabel('NO')

plt.scatter(matrix['NO'], matrix['Benzene'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('NO')
plt.ylabel('Benzene')


plt.scatter(matrix['PM2.5'], matrix['Benzene'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('PM2.5')
plt.ylabel('Benzene')
plt.scatter(matrix['CO'], matrix['O3'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('CO')
plt.ylabel('O3')

plt.scatter(matrix['Benzene'], matrix['Xylene'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('Benzene')
plt.ylabel('Xylene')

plt.scatter(matrix['Benzene'], matrix['Toluene'], c= answer, s =50, cmap = 'plasma')
center = kmeans.cluster_centers_
plt.scatter(center[:,0], center[:,1], c = 'red', s =200, alpha = 0.5)
plt.xlabel('Benzene')
plt.ylabel('Toluene')
```

INTERESTING FINDINGS IN AQI DURING COVID RESTRICTIONS

Converting "Date" into **datetime** format.
Extracting "Year" and "Month" from the "Date" variable and adding it to the main data.

```
#Extracting year from DateTime format
m2["Date"] = pd.to_datetime(m2['Date'])
m2["Year"] = m2["Date"].dt.year
m2["Month"] = m2["Date"].dt.month

#Converting datatype for Year and Month
m2 = m2.astype({"Year":object})
m2 = m2.astype({"Month":object})
```

Comparison of AQI over the years

```
pollutants = ['NO2','NH3','CO','SO2','O3']

m2.groupby('Year')[pollutants].mean().plot(figsize=(12,6), cmap='icefire')
plt.legend(bbox_to_anchor=(1.0, 1.0))
plt.xticks(np.arange(12), mth_dic.values())
```

Comparison between the AQI for the entire 2019 year and the first six months of 2020 for four major metropolitian cities of "Ahmedabad", "Bengaluru", "Delhi" and "Mumbai".

```
cities = ['Ahmedabad','Delhi','Bengaluru','Mumbai']

filtered_m2 = m2[m2['Date'] >= '2019-01-01']
AQI = filtered_m2[filtered_m2.City.isin(cities)][['Date','City','AQI','AQI_Bucket','NO2','NO','NOx']]
AQI.head()

yearly_AQI = filtered_m2[filtered_m2.City.isin(cities)][['Date','City','AQI','AQI_Bucket','Year']]
chart = sns.catplot(x="City", y="AQI", hue="Year", data=yearly_AQI, height=4, aspect=1.2,
kind="bar", palette="inferno");
chart.set_xticklabels(rotation=45);
```

Comparison of AQI before and after the lockdown

```
AQI_pivot = AQI.pivot(index='Date', columns='City', values='AQI')

from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
```

```
    rows=4, cols=1,
    #specs=[[{}, {}],
        # [{"colspan": 6}, None]],
    subplot_titles=("Ahmedabad","Bengaluru","Delhi","Mumbai"))

fig.add_trace(go.Bar(x=AQI_pivot.index, y=AQI_pivot['Ahmedabad'],
                     marker=dict(color=AQI_pivot['Ahmedabad'],coloraxis="coloraxis1")),
              1, 1)
fig.add_trace(go.Bar(x=AQI_pivot.index, y=AQI_pivot['Bengaluru'],
                     marker=dict(color=AQI_pivot['Bengaluru'], coloraxis="coloraxis1")),
              2, 1)
fig.add_trace(go.Bar(x=AQI_pivot.index, y=AQI_pivot['Delhi'],
                     marker=dict(color=AQI_pivot['Delhi'], coloraxis="coloraxis1")),
              3, 1)
fig.add_trace(go.Bar(x=AQI_pivot.index, y=AQI_pivot['Mumbai'],
                     marker=dict(color=AQI_pivot['Mumbai'], coloraxis="coloraxis1")),
              4, 1)

fig.update_layout(coloraxis=dict(colorscale='Temps'),showlegend=False,title_text="AQI Levels")

fig.update_layout(plot_bgcolor='white')

fig.update_layout( width=800,height=800,shapes=[
    dict(
    type= 'line',
    yref= 'paper', y0= 0, y1= 1,
    xref= 'x', x0= '2020-03-25', x1= '2020-03-25'
   )
])
fig.show()
```

Comparison of NO2 concentration for the years 2019 and 2020

```
new_2020 = m2.loc[m2["Year"] == 2020]
new_2019 = m2.loc[m2["Year"] == 2019]
new = pd.concat([new_2019, new_2020])

AQI_new = new[new.City.isin(cities)]
AQI_new.info()

chart = sns.catplot(x="City", y="NO2", hue="Year", data=AQI_new, height=5, aspect=1.1,
kind="bar", palette="muted");
chart.set_xticklabels(rotation=45);
```